

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA5117

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.* CO (BL4 , BL5)

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A CASE STUDY

Code of Conduct:

I Daniel Jesuraji J (192565085) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Annexure A

Case Study

1. A software company is using MD5 to store user passwords in its database. After a security breach, it is discovered that attackers were able to crack multiple passwords. Analyze the weaknesses of MD5 in this scenario and propose a more secure hashing approach using SHA-based techniques.

1. Analysis of the Problem

MD5 (Message Digest Algorithm 5) is a cryptographic hash function that produces a **128-bit hash value**. Although it was widely used in the past, MD5 is no longer considered secure for password protection.

The attackers were able to crack multiple passwords because MD5 has several important weaknesses:

a) Very Fast Hashing:

MD5 is designed to calculate hashes very quickly. This becomes a major weakness for password storage because attackers can test a very large number of possible passwords in a short time using modern hardware.

b) Collision Vulnerability:

MD5 is vulnerable to collision attacks, where different inputs can be deliberately constructed to produce the same hash. Therefore, it should not be used for security-sensitive cryptographic applications.

c) No Salt:

If the company stores passwords using only MD5(password), identical passwords produce identical hash values. Attackers can use precomputed hash databases and rainbow tables to identify common passwords.

d) Weak Passwords Remain Vulnerable:

Even if MD5 is a one-way hash, simple passwords such as common words or short combinations can be efficiently guessed and compared with their MD5 hashes.

Therefore, the breach demonstrates that **MD5 is unsuitable for securely storing passwords**.

2. Proposed Secure SHA-Based Approach

A better approach is to use a modern password-hashing scheme based on the SHA family, such as **PBKDF2-HMAC-SHA-256**, rather than applying SHA-256 directly to the password.

The recommended process is:

User Password

↓

Generate a unique random Salt

↓

PBKDF2 using HMAC-SHA-256

↓

Large number of iterations

↓

Derived Password Hash

↓

Store Salt + Hash + Parameters

For example:

Password + Unique Salt → PBKDF2-HMAC-SHA-256 → Derived Key

Each user's password should have a **unique random salt**. The salt does not need to be secret and can be stored alongside the derived hash.

During login:

Entered Password + Stored Salt → PBKDF2-HMAC-SHA-256 → Compare with Stored Hash

If the generated hash matches the stored hash, authentication is successful.

3. Why This Approach Is More Secure

Salt: Prevents identical passwords from producing identical stored values and makes precomputed rainbow-table attacks much less effective.

HMAC-SHA-256: Provides a strong cryptographic primitive for the password-based key derivation process.

High Iteration Count: Makes each password-guessing attempt computationally expensive, significantly slowing brute-force and dictionary attacks.

Unique Salt for Every User: Ensures that even users with the same password have different stored password records.

Parameter Storage: The system should store the salt and the parameters required for verification, such as the iteration count, so that the password can be verified correctly.

Conclusion

The use of MD5 for password storage was a major security weakness because MD5 is fast, vulnerable to collisions, and unsuitable for modern password protection. The company should migrate to a **salted, computationally expensive password-hashing scheme such as PBKDF2-HMAC-SHA-256**, using a unique random salt and an appropriately high iteration count. This makes password-cracking attacks significantly more difficult and provides a much stronger foundation for secure password storage.

2. A banking application implements digital signatures for transaction verification, but users report unauthorized transactions being approved. Analyze the possible flaws in the digital signature implementation and recommend improvements based on secure signature standards.

1. Analysis of the Problem

Digital signatures are used to provide **authentication, integrity, and non-repudiation** for banking transactions. If unauthorized transactions are being approved, the digital signature system may have weaknesses in key management, signature verification, or certificate validation.

The possible flaws include:

a) Private Key Compromise:

If an attacker's access to a user's private signing key is possible, the attacker can generate valid signatures and make unauthorized transactions appear legitimate.

b) Weak Key Management:

Poor generation, storage, or protection of private keys can expose them to unauthorized users or malware.

c) Improper Signature Verification:

The banking application may fail to correctly verify the signature before approving a transaction. It must verify both the signature and the exact transaction data associated with it.

d) Lack of Certificate Validation:

If the application does not properly validate the signer's digital certificate, an attacker may use an invalid, expired, revoked, or unauthorized certificate.

e) Weak Cryptographic Algorithms:

Using outdated or weak signature algorithms can reduce the security of the system.

f) Transaction Data Not Properly Bound to the Signature:

The signature must cover critical transaction information such as the **account, amount, beneficiary, and transaction details**. If these details can be changed after signing, an attacker may modify the transaction without invalidating the signature.

2. Recommended Improvements

The banking application should implement the following security measures:

1. Use Strong Digital Signature Standards:

Use modern, well-established digital signature algorithms such as **RSA with appropriate key sizes or ECDSA/EdDSA**, according to the organization's security requirements and supported standards.

2. Protect Private Keys:

Private signing keys should be securely generated and stored using protected cryptographic storage such as a **Hardware Security Module (HSM)** or secure key-management system. Private keys should never be exposed in application logs or ordinary storage.

3. Use Strong Key Management:

Implement secure key generation, distribution, rotation, backup, revocation, and destruction procedures.

4. Verify the Complete Transaction:

The application should digitally sign the complete transaction data. Before approval, it must verify that the signed information exactly matches the transaction being executed.

5. Validate Digital Certificates:

The system should check certificate validity, issuer trust, expiration, and revocation status. Certificate-chain validation should also be performed where applicable.

6. Use Secure Hash Functions:

The signature process should use a modern cryptographic hash function such as **SHA-256 or stronger**, rather than obsolete hash algorithms such as MD5 or SHA-1.

7. Add Multi-Factor Authentication:

For high-value or unusual transactions, digital signatures should be combined with additional authentication factors such as OTP or another approved second factor.

8. Implement Transaction Monitoring:

The banking system should monitor unusual transaction patterns and apply additional verification for suspicious transactions.

3. Secure Transaction Verification Flow

Transaction Created

↓

User Authenticated

↓

Transaction Details Prepared

↓

Hash of Complete Transaction Generated

↓

User's Private Key Creates Digital Signature

↓

Bank Receives Transaction + Signature

↓

Certificate and Signature Validated

↓

Signed Transaction Data Compared with Actual Transaction

↓

Transaction Approved Only if All Checks Succeed

Conclusion

The unauthorized transactions may result from **private-key compromise, weak key management, incorrect signature verification, inadequate certificate validation, weak algorithms, or failure to bind the signature to the complete transaction data**. The banking application should use strong digital signature standards, secure private-key storage, SHA-256 or stronger hashing, proper

certificate validation, complete transaction-data verification, and additional authentication for sensitive transactions. These measures strengthen **authentication, integrity, and non-repudiation** and significantly reduce the possibility of unauthorized transactions being approved.

3. An enterprise network uses Kerberos for authentication, but attackers successfully perform a replay attack to gain unauthorized access. Examine how this attack could occur and propose enhancements in the authentication process to prevent such threats.

1. Analysis of the Replay Attack

Kerberos is a network authentication protocol that uses **tickets, secret keys, and timestamps** to provide secure authentication between users and network services.

A replay attack occurs when an attacker captures a previously valid authentication message or authentication information and sends it again later to gain unauthorized access.

A typical attack can occur as follows:

1. A legitimate user authenticates with the Kerberos Authentication Server.
2. Kerberos provides the user with a valid ticket and authentication information.
3. The attacker captures authentication-related network traffic.
4. The attacker attempts to reuse the captured information to access a protected service.
5. If the system does not properly validate freshness, timestamps, or ticket information, the replayed authentication may be accepted.
6. The attacker can therefore gain unauthorized access without knowing the user's password.

The main weakness is **failure to properly detect and reject previously used or outdated authentication information**.

2. Reasons for the Attack

The replay attack may occur because of:

- Improper timestamp validation.
- Excessively large clock-skew tolerance.

- Weak or incorrect implementation of authenticators.
- Failure to detect duplicate authentication requests.
- Poor protection of Kerberos tickets.
- Compromised client systems.
- Incorrect configuration of the Kerberos Key Distribution Center (KDC).

3. Recommended Enhancements

The enterprise should implement the following security measures:

a) Timestamp-Based Authentication

Kerberos authenticators should contain timestamps. The server should verify that the timestamp falls within an acceptable time window. Old authentication requests should be rejected.

b) Clock Synchronization

All systems participating in Kerberos should maintain synchronized clocks using a trusted time-synchronization mechanism. This is important because Kerberos relies heavily on timestamps to identify stale authentication information.

c) Replay Detection

The Kerberos service should detect repeated authenticators and reject authentication requests that have already been processed.

d) Secure Ticket Management

Kerberos tickets should be protected from unauthorized access. Ticket lifetimes should be appropriately configured so that stolen tickets have a limited period of usefulness.

e) Strong Encryption and Key Protection

Kerberos keys and authentication information must be securely protected. Strong, modern encryption algorithms should be used, and long-term secret keys should not be exposed.

f) Secure Client Systems

Client machines should be protected against malware and credential theft because an attacker who compromises a legitimate client may obtain valid Kerberos credentials.

g) Proper KDC Configuration

The Key Distribution Center should be securely configured and monitored. Access controls, logging, auditing, and appropriate ticket policies should be implemented.

4. Improved Authentication Flow

User Requests Authentication



Kerberos KDC Validates User



Ticket-Granting Ticket (TGT) Issued



Service Ticket Requested



Authenticator Contains Fresh Timestamp



Service Validates Ticket + Authenticator



Timestamp and Replay Checks Performed



Valid and Fresh → Access Granted

Invalid/Repeated/Expired → Access Denied

Conclusion

The Kerberos replay attack occurs when an attacker captures valid authentication information and attempts to reuse it to gain unauthorized access. The risk can be reduced by enforcing **timestamp validation, clock synchronization, replay detection, secure ticket management, strong encryption, KDC security, and proper client protection**. These measures ensure that old or duplicated authentication information is rejected and strengthen the overall security of the Kerberos authentication process.

4. A secure communication system uses a simple hash function to verify message integrity, but attackers are able to modify messages without detection. Analyze why this approach fails and propose a solution using HMAC to improve message authentication.

1. Analysis of the Problem

A cryptographic hash function generates a fixed-length hash value from a message. It can be used to detect accidental changes to data. However, using a **simple hash alone does not provide message authentication** because the hash does not contain a secret key.

For example:

Original Message → Hash Function → Hash Value

If an attacker modifies the message, they can calculate a new hash for the modified message and replace the original hash with the newly calculated one.

Therefore:

Modified Message → Hash Function → New Hash Value

When the receiver calculates the hash, it matches the attacker's new hash, so the modification may go undetected.

The major weakness is that **both the sender and attacker can calculate the hash because no secret key is required.**

2. Proposed Solution: HMAC

The system should replace the simple hash mechanism with **HMAC (Hash-based Message Authentication Code)**.

HMAC combines:

- A cryptographic hash function
- A secret key
- The message

A simplified representation is:

Message + Secret Key → HMAC-SHA-256 → Authentication Code

The sender generates an HMAC using a secret key shared with the receiver and sends:

Message + HMAC

The receiver uses the same secret key to independently calculate the HMAC and compares it with the received HMAC.

3. HMAC Authentication Process

Sender:

Message + Secret Key

↓

HMAC-SHA-256

↓

HMAC Value

↓

Send Message + HMAC

Receiver:

Received Message + Shared Secret Key

↓

HMAC-SHA-256

↓

Calculated HMAC

↓

Compare with Received HMAC

If HMACs Match → Message Accepted

If HMACs Do Not Match → Message Rejected

4. Why HMAC Is More Secure

a) Secret Key:

Unlike a simple hash, HMAC requires a secret key. An attacker who does not know the key cannot generate a valid HMAC for a modified message.

b) Message Integrity:

If the message is modified during transmission, the calculated HMAC will not match the original HMAC.

c) Message Authentication:

A valid HMAC provides assurance that the message was generated by a party possessing the shared secret key.

d) Protection Against Message Modification:

An attacker cannot simply modify the message and calculate a valid authentication code without knowing the secret key.

e) Strong Hash Function:

Using HMAC with a modern hash function such as **SHA-256** provides a strong authentication mechanism.

Conclusion

A simple hash function is insufficient for message authentication because it does not use a secret key. An attacker can modify the message and generate a new

hash value. The communication system should therefore use **HMAC with a strong hash function such as SHA-256**. HMAC provides both **message integrity and authentication**, ensuring that unauthorized modifications can be detected and rejected.

5. An organization implements X.509 certificates for secure communication, but users encounter frequent trust validation errors. Analyze the possible issues in certificate management and propose solutions to ensure proper authentication and trust chain validation.

1. Analysis of the Problem

X.509 certificates are used to establish **identity, authentication, and trust** in secure communication systems. A certificate binds an entity's identity to a public key and is issued by a trusted **Certificate Authority (CA)**.

Frequent trust validation errors indicate that there may be problems with certificate issuance, configuration, expiration, revocation, or the certificate trust chain.

The possible issues include:

a) Expired Certificates:

A certificate is valid only for a specified period. If the certificate has expired, the client will reject it.

b) Incorrect Certificate Chain:

An X.509 certificate may be issued by an intermediate CA rather than directly by a root CA. If the required intermediate certificates are missing or incorrectly configured, the client may be unable to build a valid chain of trust.

c) Untrusted Certificate Authority:

If the issuing CA is not present in the client's trusted CA store, the certificate may be reported as untrusted.

d) Invalid or Incorrect Domain/Identity:

The identity in the certificate must match the identity of the server or entity being authenticated. A mismatch can cause certificate validation to fail.

e) Revoked Certificates:

A certificate may have been revoked by the CA before its expiration date. The system should check the certificate's revocation status.

f) Incorrect System Time:

If the client's date and time are incorrect, a valid certificate may incorrectly appear to be expired or not yet valid.

g) Poor Certificate Management:

Failure to renew certificates, update trusted CA certificates, or properly deploy intermediate certificates can result in repeated validation errors.

2. Proposed Solutions

The organization should implement the following certificate-management practices:

1. Maintain a Trusted CA Hierarchy

The organization should maintain a properly configured trust hierarchy:

Root CA

↓

Intermediate CA

↓

End-Entity/Server Certificate

The client should be able to construct and validate this complete chain.

2. Install and Maintain Trusted Root Certificates

Only appropriate and trusted root CA certificates should be installed in the organization's trusted certificate stores. Trust stores should be maintained and updated securely.

3. Deploy Intermediate Certificates Correctly

Servers should provide the required intermediate certificates so that clients can construct the certificate chain from the end-entity certificate to a trusted root CA.

4. Monitor Certificate Expiration

Organizations should maintain an inventory of certificates and monitor their expiration dates. Certificates should be renewed before expiration to prevent service interruptions.

5. Check Certificate Revocation

Certificate revocation should be checked using appropriate mechanisms such as **Certificate Revocation Lists (CRLs)** or **Online Certificate Status Protocol (OCSP)** where applicable.

6. Verify Certificate Identity

The certificate's identity information should correctly correspond to the server or service being accessed. This prevents authentication of an unintended entity.

7. Synchronize System Clocks

All systems should maintain accurate date and time settings because certificate validity depends on time.

8. Automate Certificate Management

Where practical, certificate discovery, renewal, deployment, and expiration monitoring should be automated. This reduces human errors and prevents certificates from unexpectedly expiring.

3. Certificate Trust Validation Process

Client Connects to Server

↓

Server Sends X.509 Certificate

↓

Check Certificate Validity Period

↓

Verify Server Identity

↓

Build Certificate Chain

↓

Verify Intermediate CA

↓

Verify Trusted Root CA

↓

Check Revocation Status

↓

All Checks Successful → Certificate Trusted

Any Check Fails → Connection/Authentication Rejected

4. Expected Benefits

A properly managed X.509 infrastructure provides:

- Reliable authentication of communicating entities.
- Secure certificate-chain validation.
- Reduced certificate-related connection failures.
- Better protection against untrusted or revoked certificates.

- Improved management of certificate expiration and renewal.
- Stronger overall trust in secure communication.

Conclusion:

Frequent X.509 trust-validation errors can result from **expired certificates, missing intermediate certificates, untrusted CAs, identity mismatches, revoked certificates, incorrect system time, and poor certificate management**. The organization should maintain a proper CA hierarchy, configure trusted roots and intermediate certificates correctly, monitor certificate expiration, perform revocation checks, verify identities, synchronize system clocks, and automate certificate management where possible. These measures ensure reliable **authentication and certificate trust-chain validation** for secure communication.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts

Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand